

SOC - Week-2

A) Neural Networks

1. 3B1B videos - 4:-

↳ What Neural Network is :-

Neuron

→ A thing that holds
a number b/w 0 & 1

→ Actually
a function

0.02 'Activation'

↳ For a 28×28 Pixel photo:-

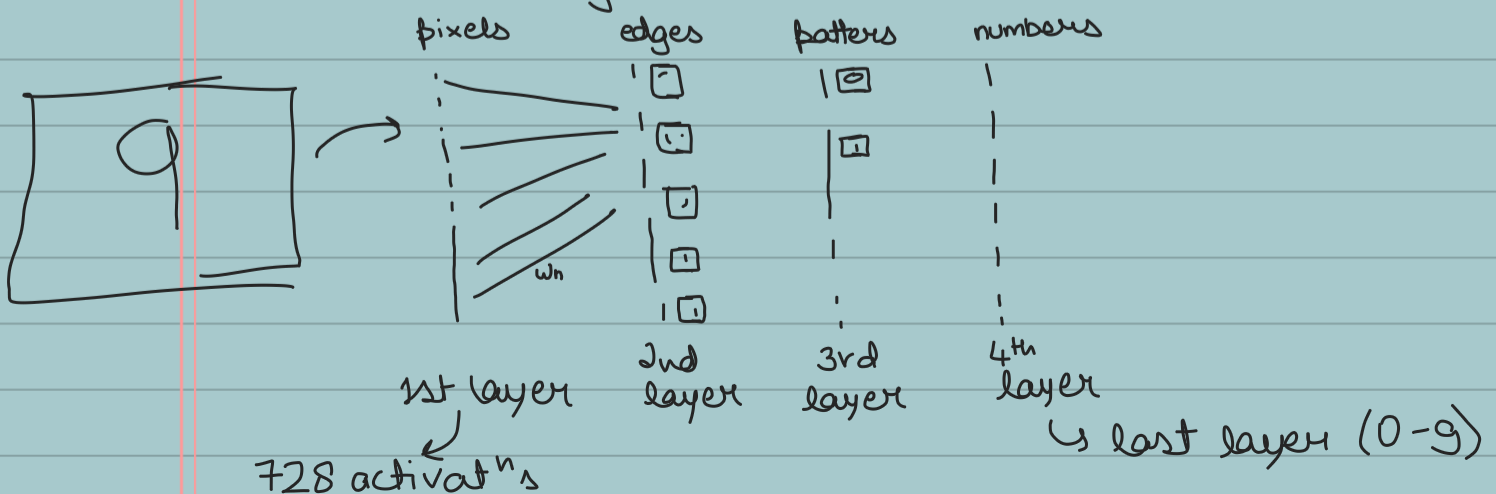
1st layer of a NN is 784 of these activation numbers
which drop down to last layer containing 10 numbers
(0 to 9)

In b/w these layers there are Hidden layers

↳ Why layers, what they do?

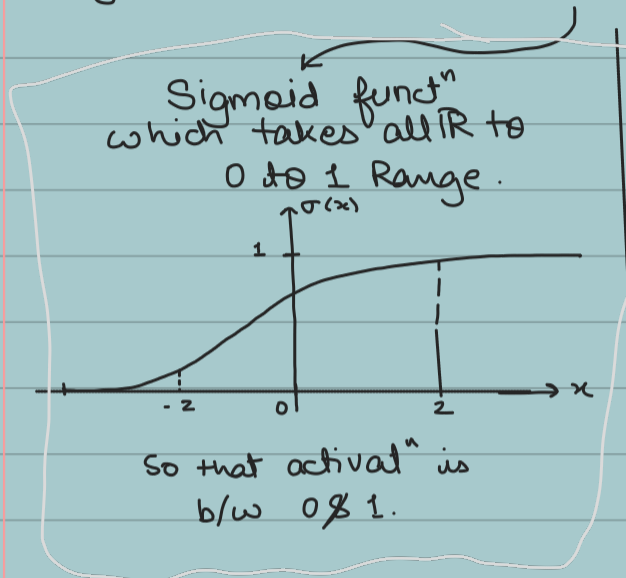
- To let say recognise a data; just like our mind connects small details and then justifies the result. different layers break down the first image data to recognise it.

- For number recognition:- (For 9)



→ Weighted Sums :-

- For a particular type of edge in the second layer to be identified we assign weight to each of the pixels.
- weights are just +ve, -ve numbers assigned to each pixel.
- to get activatⁿ we take :- $\sigma(w_1 a_1 + w_2 a_2 + w_3 a_3 + \dots + w_n a_n - x)$



what if we want that the activatⁿ is meaningful only when it is greater than 10 ; now we want bias ness which would be done by keeping $x=10$
 'Bias' for inactivity

- activatⁿ for the next layer can be written in matrix multiplicatⁿ as :-

$$\begin{array}{ccc}
 & \text{1st layer} & \text{next layer} \\
 & \downarrow & \downarrow \\
 \begin{bmatrix} w_{0,0} & \dots & w_{0,n} \\ w_{1,0} & \dots & w_{1,n} \\ \vdots & & \vdots \\ w_{m,0} \end{bmatrix} & \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ a_2^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} & = \begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_m^{(1)} \end{bmatrix} \\
 \underbrace{\hspace{10em}}_W & \underbrace{\hspace{5em}}_{A^{(0)}} &
 \end{array}$$

$$\Rightarrow a^{(1)} = \sigma(Wa^{(0)} + b)$$

↪ matrix of bias

$L \rightarrow$ Cost function:-

- To find all the values of weights and biases for a comp to work perfectly,

first, we assign random values to it; then we match expected output to the output we want.

Avg of $\boxed{\sum (\text{subtracted})^2}$ = cost function (w)
↳ minimise this

We are searching for $[-\nabla C(\dots)] \rightarrow$ neg. gradient
This will tell how we should change weights & biases.

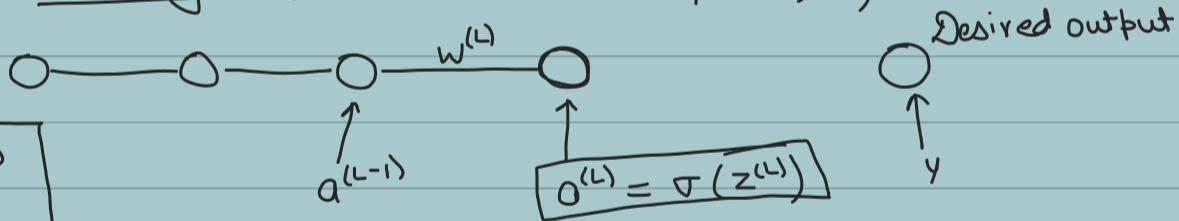
→ Back propagation:- → algo for determining how a single training ex. would change weights & biases

For single layer:- (one neuron per layer)

$$C_0 = (a^{(k)} - y)^2$$

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)}$$

$$a^{(L)} = \sigma(z^{(L)})$$



$$\omega^{(L)} \quad a^{(L-1)} \quad b^{(L)}$$

$7(4)$

9(4)

C_0

$$\frac{dC_0}{d\omega^{(L)}} = \underbrace{\frac{\partial z^{(L)}}{\partial \omega^{(L)}}}_{a^{(L-1)}} \cdot \underbrace{\frac{\partial a^{(L)}}{\partial z^{(L)}}}_{\sigma'(z^{(L)})} \cdot \underbrace{\frac{\partial C_0}{\partial a^{(L)}}}_{2(a^{(L)} - y)}$$

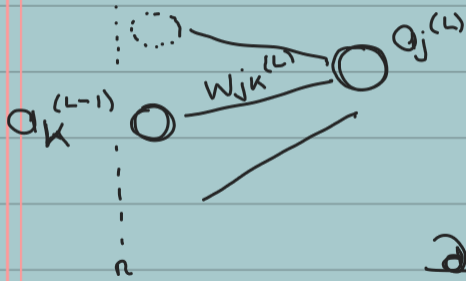
$$\frac{dC_0}{db^{(L)}} = \frac{\partial z^{(L)}}{\partial b^{(L)}} \cdot \frac{\partial a^L}{\partial z^L} \cdot \frac{\partial C_0}{\partial a^L}$$

$$\frac{\partial C}{\partial \omega^{(L)}} = \frac{1}{n} \sum_{k=0}^{n-1} \frac{\partial C_k}{\partial \omega^{(L)}}$$

Avg of
all training
example.

full cost
derivative

For multiversion layers:-



$$z_j^{(L)} = \sum_{i=0}^n w_{ji}^{(L)} a_i^{(L-1)} + b_j^{(L)}$$

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

$$\frac{\partial C_0}{\partial a_k^{(L-1)}} = \sum_{j=0}^{n_{L-1}} \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \cdot \frac{\partial C_0}{\partial a_j^{(L)}}$$